Task 1 – Serial Code - Finding Prime Numbers Write a serial C program to search for prime numbers that are strictly less than an integer n, provided by the...

PASTED

you have three tasks.
for task 1 we want to use a prime calculation function, with the following Approach:

- If k = m * n where m and n > 1
- Both factors cannot be > sqrt(k) because their product will be > k
- So at least 1 factor is <= sqrt(k)
- So testing divisors from 2 till sqrt(k) is sufficient to decide primality
- Reduces work from O(k) to O(sqrt(k))

for task 2, use the following approach using the same prime calculation function from task 1:

- Thread t tests the candidates k = 3+2t, 3+2t+2T, 3+2t+4T, ... where T is the number of threads.
- 2 is the only even prime so thread 0 records it directly and every thread then only tests odd candidates
- Done so that every thread tests a possible candidate rather than even numbered threads testing even numbers which none are primes

for task 3, use the following approach:

- Use the same prime calculation function as task 1 as there is no shared state
- Use OpenMP library and avoid using any locks

Show less

6 minutes ago

Ran 7 commands, created 3 files · 2 notes

All edge cases pass cleanly — no crashes, correct counts even with wildly over-provisioned thread counts, and the stdout/file boundary at n=100 works as intended. Now let me add a Makefile and a short README, then hand everything over.

Created 2 files, ran 2 commands, read a file · 1 note

All three programs are built, compiled clean with `-Wall -Wextra`, and verified: outputs match byte-for-byte across serial, pthreads and OpenMP for n up to 10,000,000 (664,579 primes, matching the known count), with correct behaviour at edge cases (n=2, n=100 boundary, thread counts far exceeding n).

A few notes on what's in there:

- **task1.c**: trial division to `sqrt(k)`, builds the list in ascending order as it goes (no separate sort needed), times with `clock_gettime`.
- **task2.c**: pthreads, cyclic odd-candidate distribution `3+2t+2tT`, thread 0 handles 2 directly, each thread writes to its own buffer, merged and sorted once after `pthread_join`.
- **task3.c**: same distribution scheme via `omp_get_thread_num()` / `omp_get_num_threads()`, each thread writes to its own indexed slot in a pre-sized array, so no critical/atomic/lock is needed anywhere in the parallel region.

The sandbox here only has a single CPU core, so I couldn't get a meaningful speedup reading for your Task 2/3 write-up — that part's noted in the README along with a table template. Run the actual benchmarks on your multi-core machine to get real numbers for the speedup discussion.